Standalone Progressive Retrieval Attention: A PyTorch Architecture for URI-Addressed Memory Expansion

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 9, 2026

Abstract

This paper describes a standalone PyTorch research prototype for Progressive Retrieval Attention (PRA), recast as an inference-time sparse approximation to full native self-attention. A trained SelfAttention checkpoint can be converted by splitting its existing Q/K/V projection, with no newly trained transport module. Selected historical token K/V is concatenated with local K/V under one shared softmax and the existing output projection; routing gists act only as an index. Exact tests show that reinserting all historical K/V reproduces the corresponding full causal-attention tail and that converted PRA logits match the source SelfAttention model when memory is disabled. A five-seed fixed-source synthetic study establishes a first content-causal native result: full-context SelfAttention reaches 100% answer-token accuracy versus 50.6% from the tail alone, and oracle native memory retains 100% through 32 splits while shuffled memory remains near chance. Five-seed HotpotQA- and QASPER-derived natural-text transport probes then recover 99.6% and 99.9% of dense context benefit at split 32 while activating 9.2% and 8.8% of token K/V; routed recovery is 63.4% and 60.2%, isolating selection as the larger remaining error. All three probes degrade under 64-way fragmentation. The prototype also retains an optional cross-attention transport used by earlier experiments. Those historical results support ordinary-language compatibility and frozen-path preservation but not causal retrieval. We add fixed-target controls, active-K/V diagnostics, Recovered Context Benefit, and explicit transport, sparsity, routing, causality, and dependency gaps. End-to-end production efficiency and unrestricted natural-language QA remain unclaimed.

1 Introduction

Progressive Retrieval Attention (PRA) is motivated by a common mismatch in long-context language modeling. Many tasks require access to large context, but the relevant evidence is sparse, hierarchical, and often discovered only after partial reasoning. A conventional decoder-only transformer receives a flat token sequence. PRA treats references as an index over model-native historical K/V and activates only selected positions. References may denote prior tokens, external objects, or an implicit prompt head; retrieval-augmented generation is therefore one possible source of memory, not the definition of PRA [7].

This paper documents the first standalone PyTorch prototype in the PRA program. The prototype is not a production long-context model. It is a controlled experimental instrument. Its goals are to make the model definition inspectable, expose reference use through explicit metadata, and support ablations that would be difficult to interpret in a large pretrained model. The contribution includes an initial empirical pilot, a five-seed corrected-router and parameter-controlled follow-up, and an evaluation design intended to make failure visible rather than merely demonstrate that a memory branch can lower loss.

The implementation consists of four main components. First, a tiny GPT-style language model provides causal decoder blocks. Second, a reference system parses tokens such as <REF_1> and maps them through an explicit local table to URI-addressed references; the prompt adapter can create a reserved implicit reference without adding a visible handle. Third, a resolver partitions each object into bounded chunks and stores full per-layer token K/V together with one or more paired projected-key/value gists. Fourth, a PRA attention layer performs hierarchical reference-then-chunk routing and inserts selected native token K/V into ordinary causal attention. An optional scaled cross-attention branch preserves earlier experiments.

The easiest way to follow the implementation is as a prepare-then-use pipeline. During preparation, each batch row resolves its own handles, encodes the resulting chunks with the same Transformer stack used for prompts, and records two representations at every PRA layer: a small gist for routing and full token K/V for reading. During use, all prompt rows pass through the Transformer together. The last query in each row ranks only that row's gists; selected chunks are materialized as variable-length memory; and their original token K/V is prepended to local K/V under one softmax. Local causal attention never disappears. The optional legacy mode instead reads the same memory through a separately normalized cross-attention residual.

2 Model Definition

2.1 Decoder Backbone

Let $x_{1:T}$ be a token sequence. The model embeds tokens and absolute positions:

$$H^{(0)} = E_{\text{tok}}(x_{1:T}) + E_{\text{pos}}(1:T). \tag{1}$$

For each layer $\ell \in \{0, \dots, L-1\}$, the implemented block is a pre-normalized residual decoder block:

$$\begin{aligned} \tilde{H}^{(\ell)} &= \mathrm{LN}_1(H^{(\ell)}), && (2)\\ A^{(\ell)} &= \mathrm{PRAttn}_\ell(\tilde{H}^{(\ell)}), && (3)\\ U^{(\ell)} &= H^{(\ell)} + A^{(\ell)}, && (4)\\ H^{(\ell+1)} &= U^{(\ell)} + \mathrm{FFN}_\ell(\mathrm{LN}_2(U^{(\ell)})). && (5) \end{aligned}$$

The feed-forward network is a two-layer MLP with GELU activation and configurable hidden width d_{ff}, defaulting to $4d$. The output head maps the final normalized hidden state to vocabulary logits. Configurable d_{ff} permits close parameter matching without changing attention width or depth.

2.2 Causal Self-Attention

For hidden states $X \in \mathbb{R}^{B\times T\times d}$, the local branch computes multi-head causal self-attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \tag{6}$$

$$O_{\text{local}} = W_O \,\mathrm{merge}\left[\mathrm{softmax}\left(\frac{QK^\top + M_{\text{causal}}}{\sqrt{d_h}}\right) V\right]. \tag{7}$$

The implementation stores a lower-triangular causal mask with maximum sequence length configured by `PRAConfig.max_seq_len`.

3 Reference Graph and Runtime State

3.1 Reference Graph

Let the external context be a directed attributed graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \tag{8}$$

where each node $v \in \mathcal{V}$ is a tuple (u, z, s, m, R) containing a stable URI u, content z, optional summary s, metadata m, and an explicit local reference table R. An edge $(v_i, a, v_j) \in \mathcal{E}$ exists only when $R_i[a] = u_j$; URI-prefix scans do not silently invent children. This local namespace makes recursive dependencies inspectable and prevents unrelated objects with similar URI prefixes from entering the graph.

For input sequence x, the reference table exposes a candidate set $\mathcal{H}(x) \subseteq \mathcal{V}$. A resolver

$$\rho(u, p, t) \to (z, s, R, m, \nu) \tag{9}$$

maps URI u, authorization policy p, and logical time or version t to a resolved object. Including p and t in the interface clarifies two systems requirements that the in-memory prototype currently simplifies: access control and freshness.

3.2 Selection and Expansion Operators

At layer ℓ, document u is partitioned into chunks $c \in \mathcal{C}_u$. Encoding produces full token memory $(K_{u,c,\ell}, V_{u,c,\ell})$ and paired gist sets $G^K_{u,c,\ell}, G^V_{u,c,\ell} \in \mathbb{R}^{G_c \times d}$. The default mean mode has $G_c = 1$: it reconstructs the projected heads into d dimensions and pools over chunk tokens,

$$g^K_{u,c,\ell} = \frac{1}{|c|} \sum_{j \in c} \mathrm{mergeheads}(K_{u,c,\ell,j}). \tag{10}$$

Last-token, explicit reference-end, and a registered GRU pooler also produce one gist. Deterministic k-means, diversity-selected prototypes, a line-topology self-organizing map, and a global/local hybrid may produce several. Multi-gist modes derive value gists from the same assignments as their key gists, preserving the regions that each routing key names. The routing query is the last valid projected query reconstructed across heads,

$$q_{t,\ell} = \mathrm{mergeheads}\left(X^{(\ell)} W_Q^{(\ell)}\right)_t. \tag{11}$$

For $g^K_{u,c,\ell,j} \in G^K_{u,c,\ell}$, the router computes

$$a_{u,c,\ell,j} = \cos(q_{t,\ell}, g^K_{u,c,\ell,j}), \qquad a_{u,c,\ell} = \mathrm{Agg}_{j=1}^{G_c}\, a_{u,c,\ell,j}, \tag{12}$$

with maximum aggregation by default and mean or log-sum-exp alternatives. Reference-level gist sets are constructed separately from chunk gist sets, allowing a true URI-first stage before chunk scoring. Hierarchical search selects at most k_r references and then at most k_c chunks independently inside each retained reference. A threshold is applied without converting the two budgets into one flattened top-k. A global-chunk strategy is available for ablation but still enforces both constraints. An expansion operator

$$\mathcal{X}(A, \mathcal{G}, b, d) \to A' \tag{13}$$

maps active set A to an expanded set A' under branching budget b and depth budget d. The corrected cache builder implements bounded child-first expansion. It follows only each resolved object's local table, shares reference and token budgets across the traversal, records build states and dependencies, and applies explicit cycle, missing-reference, depth, and overflow policies. This deterministic expansion is a runtime policy, not yet a learned iterative controller.

3.3 Cache and Lifecycle State

The runtime state before token t is

$$\Omega_t = (\mathcal{G}, \mathcal{H}_t, A_t, \mathcal{C}_t, p_t, \nu_t), \tag{14}$$

where A_t is the active reference set, $\mathcal{C}_t$ the resident encoded cache, p_t policy state, and ν_t provenance/version state. A cache entry is keyed conceptually by $(u, v, \ell, m, \theta, p)$: URI, content version, layer, model identity, relevant encoder parameters, and policy domain. The prototype key is only URI because each cache is rebuilt per example; persistent or shared caches require the fuller identity.

One runtime transition can be written

$$\begin{aligned} A_t &= \mathcal{S}_{k,\tau}(q_t, \mathcal{C}_t), && (15)\\ A'_t &= \mathcal{X}(A_t, \mathcal{G}, b, d), && (16)\\ \mathcal{C}'_t &= \mathrm{admit}(\mathrm{encode}(\rho(A'_t))), && (17)\\ (y_t, \Omega_{t+1}) &= \mathrm{attend}(x_{\le t}, \mathcal{C}'_t, \Omega_t). && (18) \end{aligned}$$

In the corrected prototype, resolution and encoding happen before the batched prompt forward. Each row's cache is ephemeral and private, and a batch-aware wrapper preserves those namespaces while the prompt itself executes once as a tensor of shape $[B, T]$. Recursive construction is available, but admission retains every successfully built entry within fixed budgets and expansion is not learned. Entries move through explicit `unseen`, `building`, `ready`, and failure states so partially built cyclic entries never become searchable. Persistent admission, eviction, and cross-request reuse remain outside the implementation.

4 Reference Handles and Resolver

4.1 Reference Tokens

The prototype uses explicit textual handles of the form `<REF_n>`. A regular expression parser extracts occurrences and resolves them only through the current object's `ReferenceTable`. Each table entry stores an id, token, URI, optional summary, and metadata. Legacy `!!ref:uri!!` syntax is retained only for compatibility.

4.2 URI and Anchor Semantics

References point to simple URIs such as `mem://doc1`. Anchored locations may use URI fragments, for example `mem://doc1#section.a`, but lexical URI ancestry is not semantic graph ancestry. The in-memory resolver stores structured document records containing text, an optional summary, metadata, version, anchors, and a local reference table. A missing URI raises an explicit error; missing-reference policy is owned by the recursive builder rather than hidden inside resolution.

4.3 Resolved References

Given a handle r with URI u, the resolver returns

$$\mathrm{resolve}(u) = (\mathrm{text}_u, \mathrm{summary}_u, R_u, \mathrm{metadata}_u, \mathrm{version}_u). \tag{19}$$

This simple resolver is sufficient for synthetic and local experiments. Future systems can replace it with file, database, vector-index, search, or web resolvers while preserving the same high-level interface.

4.4 Implicit Prompt-Head Reference

The same resolver/cache contract can retain prompt history that no longer fits the local window. Let N be the exact input-token count and let

$$D = \min(\texttt{max\_prompt\_direct\_tokens}, \texttt{max\_seq\_len}), \tag{20}$$

where an unset direct limit inherits `max_seq_len`. In `implicit_reference` mode, an input with $N > D$ is split into head IDs $x_{1:N-D}$ and direct-tail IDs $x_{N-D+1:N}$. The adapter registers the head under the reserved URI `pra://implicit/prompt/head` and diagnostic handle `#__head`, then passes its token IDs directly to ordinary chunking, cache construction, gist routing, and selected-memory attention. It does not decode and retokenize the head or insert a synthetic marker into the visible tail.

Each batch row owns its implicit object even though all rows reuse the same URI string. Mixed tail lengths are padded with a validity mask, and the routing query is taken from the last valid position rather than the final rectangular column. Explicit references remain in the same row-private cache. The prompt head may use a separate chunk cap via `max_prompt_gists`; `None` retains every prompt-head chunk, while chunk length itself remains bounded by the model context limit. The alternative overflow policies preserve the prior behavior: `truncate` keeps only the recent tail and `error` rejects the request.

Preparation reports the total, direct, implicit, chunk, and gist counts together with whether overflow occurred. The reserved object is invalidated between requests, and an explicit reference cannot claim its URI. Generation prepares the initial prompt through this path, but the current prototype does not periodically move newly generated tokens into an implicit head. Thus it supports long initial prompts, not unbounded streaming generation.

5 Layer-Specific Memory Cache

5.1 Cache Entries

The PRA cache stores a reference entry containing layer-specific chunk memory:

$$C_u = \Big(u, z_u, m_u, \big\{\big(c, K_{u,c,\ell}, V_{u,c,\ell}, G^K_{u,c,\ell}, G^V_{u,c,\ell}\big)\big\}_{c,\ell}\Big). \tag{21}$$

Each chunk records a deterministic identifier, source URI, token interval, metadata, full projected token K/V, and a layer-specific paired gist set. K/V and gists are detached by default; a trainable-gist mode preserves the registered GRU gist path. Optional summary evidence, when present, is projected contextually into the same layer space and attached to the chunk gist set. The implementation exposes an abstract `PRAMemoryCache` and a dictionary-backed `PRASimpleMemoryCache`.

5.2 Reference Encoding Pass

Before a batch forward pass, the training code builds isolated caches from batch metadata. For each referenced document:

1. Resolve the URI to text, optional summary, metadata, and its explicit local reference table.
2. Recursively ensure children are ready first when recursive references are enabled, subject to shared depth, reference, and token budgets.
3. Partition the text with `none`, bounded fixed windows, explicit markers, or a supplied semantic partitioner. Semantic mode without a plugin fails explicitly.
4. Run each document through the same model path. Parent encoding may read already-ready child memory; nonrecursive encoding disables PRA memory.
5. At every PRA layer, project each chunk's token states through that layer's W_K and W_V, then derive paired key/value gist sets under the configured strategy.
6. Store provenance, fingerprints, truncation details, layer-specific chunk tensors, and dependency events before atomically marking the entry ready.

The fourth and fifth steps are the important representation boundary. Reference text is not mapped directly to a single static document vector. A chunk starts at the embedding layer, advances through the model, and exposes the K/V that the corresponding PRA layer would consume. Consequently a lower-layer cache and a higher-layer cache describe the same source at different model depths. One or more routing gists are then pooled from the projected keys at that depth. They are suitable for inexpensive ranking; they do not replace the detailed token memory.

Fixed windows can overlap, but selected materialization deduplicates overlapping source-token spans. Maximum gist counts and overflow policies bound routing state. Cache fingerprints include content, model, tokenizer, and routing identities so a persistent implementation can invalidate stale representations. The current per-example cache remains ephemeral, but the identity rules are explicit.

6 PRA Attention

6.1 Hierarchical Content Routing

At generation or training time, each batch row independently compares its projected last-valid-token query with layer-specific chunk gists. For query $q_{b,t,\ell}$ and chunk gist set $G^K_{u,c,\ell}$, the default content score is

$$\text{score}(u, c \mid q) = \max_{g \in G^K_{u,c,\ell}} \frac{q^\top g}{\|q\|_2 \|g\|_2}. \tag{22}$$

This score is a coarse admission decision: it asks which chunks are worth opening for the current row and layer. It is distinct from the token-level attention weights computed after a chunk is opened. The default hierarchical policy uses independent reference and per-reference chunk top-k settings. They are configured as `top_k_references` and `top_k_chunks_per_reference`; either may be zero, yielding no memory. `top_k_refs` remains a warning-producing compatibility alias for one release. Summaries are disabled by default. If enabled, `replace`, `hybrid`, and `augment` policies combine projected summary evidence with content scores; a missing summary always falls back to content routing.

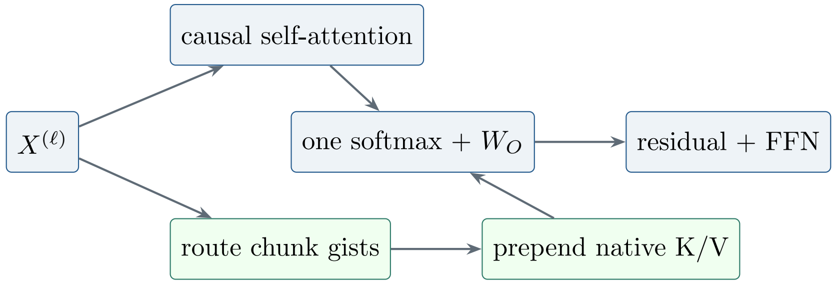

Figure 1: Canonical PRA block. Selected native K/V and local K/V share one normalization and the existing output projection.

6.2 Canonical Native-KV Transport

For selected chunk set $\mathcal{A}_{b,\ell}$ and layer ℓ, the implementation materializes only selected full-token memories for each batch row:

$$K_{b,\ell}^{\text{mem}} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} K_{u,c,\ell}, \quad V_{b,\ell}^{\text{mem}} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} V_{u,c,\ell}. \tag{23}$$

It then prepends memory K/V to local K/V and computes one causal normalization:

$$O = W_O \,\text{merge}\left[\text{softmax}\left(\frac{Q_b[K_{b,\ell}^{\text{mem}}; K_{b,\ell}^{\text{local}}]^\top}{\sqrt{d_h}} + M_{\text{causal}}\right)[V_{b,\ell}^{\text{mem}}; V_{b,\ell}^{\text{local}}]\right]. \tag{24}$$

Memory positions are visible to every local query; local positions retain their ordinary causal and padding masks. There is no $W_{O,\text{mem}}$, separate softmax, or memory scale. If memory is disabled or selection is empty, the implementation takes the exact local path. Routing gists index token K/V and are not themselves transported, except in the explicit `gist_only` diagnostic.

The normal `selected_chunks` mode transports exactly the token K/V belonging to the routed chunks. Two diagnostic modes make the boundary testable: `full_reference` opens every chunk belonging to a selected URI, while `gist_only` supplies one K/V position per selected chunk. The first removes chunk selection from the experiment; the second asks how much can be done without token-level detail. Neither changes the URI or reference-ranking stage.

Variable memory lengths are executed in one of three modes. Mode zero evaluates each nonempty row independently; mode one pads a single masked rectangle; and mode $K \geq 2$ deterministically partitions rows into at most K contiguous length buckets using either equal-count grouping or a dynamic-programming minimum-padding plan. Masks preserve numerical equivalence, empty rows remain zero, and outputs are restored to original batch order. Diagnostics report actual bucket count, selected versus padded positions, and allocation ratio.

6.3 Optional Adapted Cross-Attention Transport

Setting `memory_transport=cross_attention` restores the earlier separately normalized branch, output projection, and residual scale. Legacy checkpoint booleans migrate to this mode, so historical reports remain reproducible. This mode changes the model and requires adaptation; it is retained as a later transport comparison rather than the definition or default of PRA.

7 Complexity and Resource Model

Let L be the number of decoder layers, L_p the number of PRA-enabled layers, T local sequence length, r the number of resolved references, m_i the token length of reference i, $M = \sum_{i \in A} m_i$ the selected resident memory length, d hidden width, and w bytes per cached scalar. Ignoring heads and constant factors, local decoder attention costs

$$O(LT^2 d). \tag{25}$$

The prototype separately encodes every resolved reference through the model, costing approximately

$$O\left(Ld\sum_{i=1}^{r} m_i^2\right) + O\left(L_p d^2 \sum_{i=1}^{r} m_i\right) \tag{26}$$

for independent reference self-attention and layer-specific K/V projection. Native memory attention adds

$$O(L_p T M d) \tag{27}$$

to a full-sequence forward pass. The resident K/V payload is approximately

$$B_{\text{cache}} = 2wL_p M d \tag{28}$$

bytes, excluding summaries, object metadata, allocator overhead, and duplicated entries. Canonical native transport adds no trainable projection. The optional cross-attention mode adds one $d \times d$ memory output projection plus bias per PRA layer.

For an overlong prompt with displaced head length $P = N - D$, the direct local term becomes $O(LD^2 d)$, while cold construction of head memory adds approximately the sum of the quadratic costs of its bounded chunks plus their K/V projections. Routing scores $G = \sum_c G_c$ small vectors, and selected-memory attention still depends on transported token count M. This decomposition is a measurement plan, not an efficiency claim: without reuse or selective materialization, separate head encoding may cost as much as or more than a strong long-context baseline.

These costs explain why shorter visible prompts alone are not a sufficient efficiency result. If every reference is encoded once per example and never reused, $\sum_i m_i^2$ can dominate. PRA becomes attractive only when selection keeps M small, encoding is amortized across tokens or requests, summaries avoid unnecessary resolution, or quality improves under a fixed compute budget. A systems evaluation must decompose resolver time, tokenization, reference encoding, selection, memory attention, local model compute, cache transfer, and synchronization.

The current batch-aware cache wrapper preserves semantic isolation and permits one $[B, T]$ prompt forward. Reference construction and row-local routing still iterate over logical rows, so they are not yet fully vectorized. A production design would also pack reference encodings and routing candidates with row offsets or masks. Its complexity can approach the same aggregate arithmetic while reducing launch and orchestration cost, provided selection and cache identity remain isolated.

8 Dataset and DataLoader Pipeline

8.1 Dataset Schema

Datasets are represented by three JSONL files per stage:

- `documents.jsonl`: URI-addressed documents, optional summaries, titles, anchors, metadata, and local reference tables.
- `references.jsonl`: reference ids, URIs, summaries, anchors, and metadata.
- `questions.jsonl`: prompts, answers, expected reference ids/URIs, expected chunk ids or spans, and expected anchors.

The loaded sample type is `QuestionSample`, containing a question, answer, list of `ReferenceSample` objects, optional reference/chunk relevance labels, and metadata. Chunk labels are optional: chunk metrics are omitted rather than fabricated when labels are absent. New dataset generators do not emit summaries; loaders retain backward compatibility with legacy optional summary fields and explicit summary experiments. The current registry includes synthetic memory QA, hierarchical synthetic references, code repositories, Wikipedia, books, technical documentation, and GitHub-style repositories.

8.2 Collation

The `PRACollator` tokenizes each question and answer, forms next-token labels, pads variable-length rows, builds a per-sample reference table, and preserves non-tensor metadata. Prompt labels are replaced by the padding/ignore id, so reference-conditioned loss is computed only over answer-continuation tokens. This metadata is essential: the training step uses it to resolve and encode references before the main model forward pass.

8.3 Data Module

The `PRADataModule` used by synthetic and repository-style stages builds a compact tokenizer from questions, answers, summaries, and reference texts. The WikiText-2 study instead trains a 2,000-token BPE tokenizer with whitespace pre-tokenization and reserves <REF_1> through <REF_5>. The serialized phase-1 tokenizer is restored for phase 2 and for post-fine-tuning plain-text evaluation. This prevents vocabulary drift from masquerading as an architectural effect.

8.4 Reference-Conditioned WikiText-2

The pilot generator retains natural WikiText language and creates *reference-conditioned continuation*, not conventional question answering. Each eligible source entry is divided into one to five earlier parts plus a tail. Earlier parts become URI-addressed references; the final 8–16 words of the tail become the prediction target. The visible prompt contains only reference handles and the neutral instruction `Continue the referenced WikiText passage:`. It therefore does not provide a local lexical prefix from which the target can be copied. Provenance records include the source entry, part boundaries and identifiers, candidate and intended references, tail span, token distance, part count, local-context sufficiency, and relation type.

That first generator labels all earlier parts as intended references and regenerates data from each run seed. We retain its measurements as a version-1 pilot. The controlled follow-up uses a version-2 dataset generated once with seed 1729 and shared across every architecture and model seed. It assigns candidate identifiers and order independently of relevance, marks one adjacent source part as the expected reference, and records source and split provenance. The fixed 512-example corpus yields 409/51/52 train/validation/test examples and a candidate-count-weighted test top-1 chance rate of 0.4202. This removes model-seed-dependent data pairing and permits

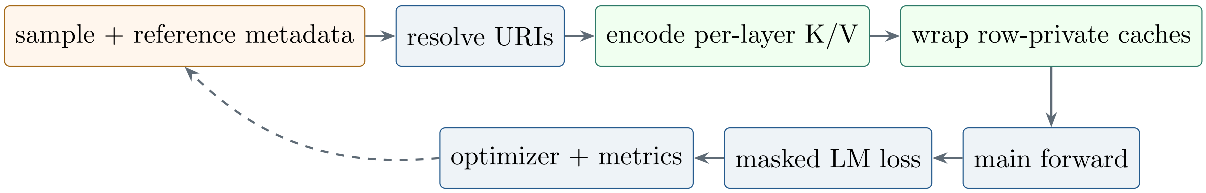

Figure 2: Implemented training/runtime pipeline. Row-private caches are constructed before one batched prompt forward; prompt labels are ignored in reference-conditioned loss.

actual selection metrics, although adjacency remains only a programmatic relevance label rather than human-verified causal evidence.

9 Training Objective and System

The training objective is standard autoregressive cross-entropy:

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_{\leq t}, C), \tag{29}$$

where C is the batch-specific PRA cache constructed from metadata. The training loop uses AdamW, optional learning-rate warmup, gradient accumulation, gradient clipping, optional mixed precision, checkpointing, early stopping, and TensorBoard/W&B/ClearML-compatible logging hooks.

The implemented PRA batch step is:

1. Move tensor fields to the target device.
2. For each example, build a fresh PRA cache from only that example's metadata without attaching it as a global cache.
3. Wrap the completed row caches in a batch-aware cache whose search maps query row b only to cache namespace b.
4. Attach that wrapper once and run one prompt forward on `input_ids` with shape $[B, T]$.
5. Compute cross-entropy with padding ignored.

Per-example cache isolation is semantically important. A previous batch-union implementation allowed each sequence to attend to references belonging to other examples and made within-batch shuffling ineffective. The corrected wrapper keeps the expensive Transformer prompt computation batched without flattening the logical address spaces. Reference-cache construction and routing are still row-wise; those are separate optimization targets from prompt-forward batching.

9.1 Native Evaluation and Historical Two-Phase Optimization

The canonical path trains an ordinary SelfAttention language model, freezes its checkpoint, converts the existing Q/K/V and output projections, and evaluates native K/V without PRA training. The historical cross-attention follow-up instead evaluates three reference-task orders: (i) training from scratch; (ii) restoring the matching phase-1 checkpoint, freezing the base, zero-initializing $W_{O,\mathrm{mem}}$, and optimizing only memory output projections; and (iii) restoring the checkpoint and directly fine-tuning all parameters jointly. Zero initialization makes the initial adapted branch an exact no-op. The frozen regime therefore preserves the pretrained local function by construction, while the measured plain-task reevaluation verifies that the implementation honors this constraint. The joint regime tests whether unconstrained adaptation is sufficient and quantifies catastrophic forgetting. These optimization findings apply to adapted cross-attention, not parameter-free native transport.

9.2 Execution-Time Accounting

The generic training engine records batch, epoch, and complete-run timing. CUDA is synchronized immediately before and after timed GPU regions so queued kernels are included. Reports contain training, validation, test, and wall-clock durations; processed tokens; sequences seen; optimizer steps; optimizer steps per second; and training tokens per second. Training duration sums synchronized batch regions, validation duration sums explicit validation passes, and wall-clock duration includes orchestration and test evaluation. These definitions should not be interchanged when comparing runtime cost.

10 Evaluation Metrics

The evaluation code reports both language-model and reference-behavior metrics:

- **Loss and perplexity**: autoregressive language-model quality.
- **Answer accuracy**: exact containment of the expected answer in generated output or decoded continuation.
- **Reference ranking**: actual selected-URI hit, recall, precision, F1, MRR, and NDCG at k, globally and per layer.
- **Chunk ranking**: selected-chunk hit, recall, precision, F1, MRR, NDCG, and optional source-span IoU against labeled chunks.
- **Expected anchor hit**: whether expected anchors appear in cached text or selected URI strings.
- **Expanded reference count**: average number of ready and recursively expanded references per example, with budget and failure events.
- **Available versus selected memory**: cache references, cache chunks/tokens, selected chunks, and actually materialized memory tokens.
- **Batching efficiency**: requested/actual bucket count, selected positions, allocated padded positions, and allocation ratio.
- **Full-context token count**: token cost of a full-context baseline.
- **Cache hit ratio**: fraction of examples whose expected references are represented in cache.
- **Latency and throughput**: wall-clock evaluation time, examples per second, and tokens per second.
- **Preservation loss**: plain WikiText-2 loss before and after reference-format fine-tuning.

- **Reference ablation loss**: answer-token loss with valid, disabled, shuffled, irrelevant, empty, and oracle reference memory.

These metrics are designed to test the PRA hypothesis directly. A useful PRA model should not merely achieve high answer accuracy; it should do so while selecting the right references and using fewer retrieved tokens than a full-context baseline.

The implemented ablation path supports disabled memory, shuffled references, irrelevant references, empty memory, oracle chunks, all selected chunks, full-reference materialization, and gist-only materialization. These are distinct interventions: cache size is not reported as selected-memory use, and a selected URI does not imply that all of its chunks were materialized. A valid-minus-disabled comparison tests whether the memory branch contributes. Valid versus shuffled or irrelevant is stricter because it asks whether the contribution depends on supplied content. Oracle chunks separate chunk routing from transport. The SelfAttention control is invariant across these conditions by construction.

10.1 Implementation Validation

The corrected mechanics are covered by focused tests rather than presented as model-quality evidence. Tests verify exact projected-key mean pooling; layer- and batch-specific routing; independent reference/chunk budgets; deterministic chunk provenance and overflow; explicit failure for unconfigured semantic chunking; optional summary fallback; GRU-gist gradients and checkpoint registration; recursive child-first build order and cycle safety; no-match equality with the local-only path; and dynamic-bucket output/gradient equivalence. A CUDA smoke test with two batch rows, different selected URIs, unequal memory lengths, and two buckets verifies finite forward/backward execution and independent selections. These checks establish implementation invariants only. The five-seed controlled study below then exercises the corrected path through training, counterfactual evaluation, retention checks, and synchronized timing; it does not by itself establish causal retrieval.

10.2 Required Systems Diagnostics

Task loss is insufficient to explain a memory system. Table 1 separates signals already emitted by the prototype from publication-grade diagnostics still required.

Table 1: Systems diagnostics and current implementation status.

Diagnostic	Measurement	Status
Reference selection	hit, recall, precision, F1, MRR, NDCG against relevant URIs	implemented globally and by layer
Chunk selection	ranking metrics and source-span IoU	implemented when labels are available
Attention behavior	routing scores, selected provenance, memory norms	structured trace implemented; heatmaps pending
Layer utilization	selected references/chunks/tokens and branch diagnostics	implemented per layer
Cache reuse	hit rate by URI/version and avoided encoding tokens	per-run hit only; persistent reuse pending
Latency	resolve/cache build, routing, materialization, memory attention, train/eval	cache and attention detail available; full resolver decomposition pending
Dynamic batching	selected versus allocated positions and bucket count	implemented with exact mode comparisons
GPU memory	peak allocated bytes and cache payload bytes	current allocation emitted; peak/decomposition pending
Failure cases	wrong URI, stale content, local insufficiency, poisoning, leakage	traces implemented; labeled taxonomy pending

Attention heatmaps should retain URI and token boundaries rather than concatenate memory into an anonymous axis. Layer utilization should distinguish a selected entry from an entry that receives meaningful attention mass. Cache reuse should count avoided encoding work, not merely dictionary hits. Failure reports should preserve prompt, target, candidate references, selected references, content versions, per-layer behavior, and the counterfactual outputs under disabled and shuffled conditions.

11 Experimental Protocol

The pilot compares three same-width architectures:

1. **SelfAttention**: four PyTorch causal SelfAttention layers.

2. **Full PRA**: four local-plus-memory PRAttention layers.

3. **Hybrid PRA**: two lower SelfAttention layers followed by two PRAttention layers.

All use four layers, four heads, width 256, context length 128, dropout 0.1, and one shared BPE tokenizer per study. The version-1 pilot uses seeds 1 and 7, batch size 2, learning rate 5×10^{-4}, 2,048 optimizer steps, and exactly 524,288 processed tokens per run. The corrected controlled follow-up uses paired seeds $\{1, 7, 21, 42, 87\}$, extends ordinary-language training to 4,096 steps and 1,048,576 tokens, fixes reference data and splitting with seed 1729, and compares 512-step scratch, frozen-reference-path, and joint reference training at learning rate 2×10^{-4}. All corrected runs execute with Python 3.10, PyTorch 2.12.1 with CUDA 12.6, and an NVIDIA GeForce GTX 950M.

The same-width parameter counts are 4,216,320 for SelfAttention, 4,479,488 for full PRA, and 4,347,904 for hybrid PRA. Full and hybrid PRA therefore contain 6.2% and 3.1% more parameters than SelfAttention. In phase 2, the PRA trainable counts are 263,168 and 131,584 because only four or two memory output projections are optimized; the SelfAttention control fine-tunes all 4,216,320 parameters. These differences delimit the claims that can be drawn from the pilot.

Table 2: Current pilot hyperparameters. Phase-2 processed tokens vary slightly with generated example lengths.

Setting	Phase 1: plain WikiText-2	Phase 2: references
Tokenizer	shared BPE, vocabulary 2,000	restored phase-1 BPE
Model	4 layers, 4 heads, width 256	matching phase-1 checkpoint
Context/batch	128 / 2	128 / 2
Optimizer budget	2,048 steps; 524,288 tokens	512 steps; mean 46,301.5 tokens
Learning rate	5×10^{-4}	2×10^{-4}
Dropout	0.1	0.1
References	absent	1–5 parts; top-$k = 5$
Threshold/α	inactive	-1.0 / 1.0
Trainable parameters	all	all SA; memory output only for PRA
Seeds	1, 7	1, 7

The parameter-matched SelfAttention controls retain width 256 and increase each four-layer FFN hidden width from 1,024 to 1,152 or 1,088. The resulting models contain 4,478,976 and 4,347,648 parameters, respectively: 512 fewer than full PRA and 256 fewer than hybrid PRA. This isolates most of the attention-branch capacity difference without changing layer count, head count, or representation width.

The evaluated ablations are:

- valid per-example references;
- a cleared and explicitly disabled PRA path;
- references deterministically shuffled across examples;
- plain WikiText-2 reevaluation after phase 2.

The corrected follow-up evaluates valid, disabled, shuffled, irrelevant, empty, and oracle-reference conditions plus parameter-matched controls in cross-attention mode. Native evaluation begins instead with `SA-full`, `SA-tail`, `PRA-native-all`, `PRA-native-oracle`, `PRA-native-disabled`, and `PRA-native-shuffled`. The report computes $\Delta_{\text{transport}} = L_{\text{native}-\text{all}} - L_{\text{SA}-\text{full}}$, $\Delta_{\text{sparse}} = L_{\text{native}-\text{oracle}} - L_{\text{native}-\text{all}}$, $\Delta_{\text{routing}} = L_{\text{native}-\text{routed}} - L_{\text{native}-\text{oracle}}$, $\Delta_{\text{memory}} = L_{\text{SA}-\text{tail}} - L_{\text{native}-\text{oracle}}$, and $\Delta_{\text{causal}} = L_{\text{native}-\text{shuffled}} - L_{\text{native}-\text{oracle}}$. Together, the first three gaps decompose the path from dense SelfAttention to all-memory transport, sparse oracle materialization, and routed selection. We also report Recovered Context Benefit for condition c,

$$\text{RCB}_c = \frac{L_{\text{SA}-\text{tail}} - L_c}{L_{\text{SA}-\text{tail}} - L_{\text{SA}-\text{full}}}. \tag{30}$$

RCB is not clipped: zero means no dense-context benefit is recovered, one means all of it is recovered, and negative values mean memory is harmful relative to the tail. It is marked undefined when the full-versus-tail denominator is numerically negligible. Active fraction is $(T_{\text{local}} +$

$T_{\text{retrieved}})/(T_{\text{local}} + T_{\text{displaced}})$, where routing gists and index vectors are recorded separately rather than counted as token K/V. The raw per-example records additionally contain target identity, selected URI/chunk sets, hit and recall signals, K/V transfer bytes, routing/materialization/attention time, and peak CUDA allocation. Reports aggregate per example, seed, and split with means, population standard deviations, medians, and approximate 95% intervals.

11.1 Native-KV Experimental Progression

The first gate converts one trained SelfAttention checkpoint and verifies disabled-memory logit equivalence. The second extracts historical K/V while processing prefix A in its original left context and reinserts all of it while processing tail B; this must match the tail of full attention with equivalent positions and masks. Independent chunk re-encoding is reported separately because reset positions and missing left context make it a different intervention. Only after all-K/V transport passes should an oracle active K/V budget frontier be measured, followed by routed and shuffled controls.

The fixed-target WikiText generator supports split counts $\{2, 3, 5, 8, 16, 32, 64\}$. For a given source and seed, it keeps the recent direct tail, answer, target identifier, and evaluation mask unchanged. Only the boundaries inside displaced prefix history vary; metadata-only implicit references avoid adding visible handle tokens. This corrects the earlier split-2/split-5 design, whose target and processed-token count changed with split count. The synthetic and evidence-anchored natural-text probes below use this invariant writer and verify the fixed target identifier before evaluation.

11.2 Publication-Grade Experiment Matrix

The current pilot is the first row of a larger preregistered-style matrix. Table 3 marks required comparisons without assigning unmeasured values.

Table 3: Required experiment matrix after the controlled follow-up. "Pending" means no empirical claim is made in this paper.

Axis	Required comparison	Status
Reproducibility	seeds $\{1, 7, 21, 42, 87\}$ with intervals/effect sizes	five paired seeds complete
Capacity	same-width plus FFN parameter-matched SA controls	complete at tiny scale
Reference causality	valid, disabled, shuffled, irrelevant, empty, oracle chunks	corrected URI-level smoke complete; chunk-level study pending
Cache budget	entries/tokens/bytes; reuse on/off; eviction policies	pending
Memory strength	α sweep including zero and learned gate	$\alpha = 1$ only
Selection	reference/chunk top-k, threshold, oracle chunks, learned router	fixed-k smoke complete; controlled sweep pending
Transport	native all/oracle/routed versus adapted cross-attention	synthetic, HotpotQA-derived, and QASPER-derived native sweeps complete
Adaptation	scratch, frozen branch, LoRA, adapters, joint fine-tuning	scratch/frozen/joint complete
Placement	full PRA, upper-layer hybrid, layer-count sweep	full and 2+2 hybrid complete
Difficulty	distance, candidate count, local sufficiency, recursion depth	recursive runtime present; experiments pending

The five-seed comparison uses a fixed tokenizer and split, equal processed-token budgets, and paired seeds. Future cache and selector sweeps should initialize from the same checkpoints. Oracle references isolate memory transport from selection; irrelevant and empty controls distinguish content use from generic branch activation. Native K/V injection must account for positional encoding and causal semantics rather than simply concatenate tensors. LoRA, adapters, and joint fine-tuning should be matched by trainable parameter count or reported as separate adaptation regimes.

12 Fixed-Source Native-KV Synthetic Result

The first native quality gate uses 2,048 training examples and a disjoint 64-example evaluation set. Every example contains the same 63 source-unit structure, a fixed historical marker whose random binary value is the target, and a one-token local query with no answer information. The seven conditions partition the identical source into 1, 2, 4, 7, 15, 31, or 63 references. Thus source text, target, model weights, and local prompt are invariant; only reference boundaries change. Five paired seeds train a two-layer, width-64 SelfAttention model for 100 optimizer steps on full context. Each checkpoint is then converted weight-for-weight to native PRA. No PRA transport or selector parameter is trained. The fixed marker position deliberately isolates transport from semantic search; random-position and learned-routing tasks are harder follow-ups.

Full-context SelfAttention obtains mean loss 0.0105 and 100% accuracy, whereas tail-only evaluation obtains loss 1.6999 and 50.6% accuracy. At split counts 3, 5, 8, 16, and 32, oracle native

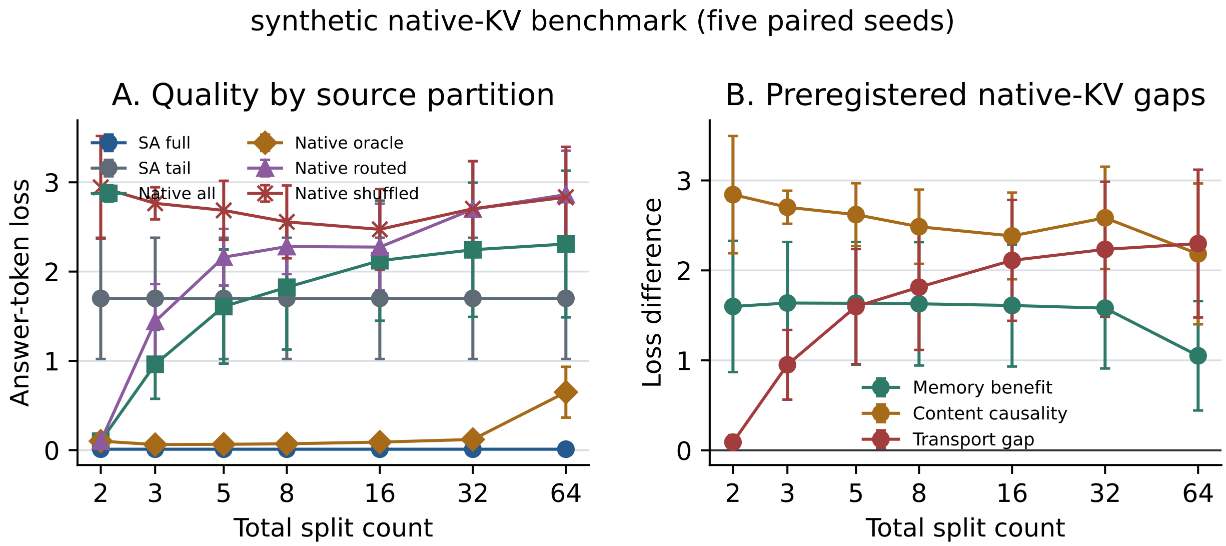

Figure 3: Five-seed fixed-source synthetic native-KV result. Full-context SelfAttention solves the dependency while the local tail is at chance. Oracle memory preserves the required content through 32 splits and sharply separates from shuffled content. Native-all and unsupervised routed conditions degrade as the same source is fragmented, distinguishing successful sparse transport from unresolved cache-composition and selection behavior. Error bars are population standard deviations.

loss is respectively 0.0624, 0.0653, 0.0711, 0.0900, and 0.1194, with 100% accuracy in every case. Shuffled losses over the same range are 2.7645, 2.6853, 2.5565, 2.4730, and 2.7038; the resulting content-causality gaps are 2.7021, 2.6200, 2.4854, 2.3831, and 2.5844. This is direct evidence that the oracle result depends on the supplied value rather than generic memory activation.

The result also identifies two limits. At 64 splits, oracle loss rises to 0.6490 and accuracy falls to 70.6%. Native-all accuracy declines from 100% at split 2 to 71.9% at split 3, 59.1% at split 5, and approximately chance by split 16. The routed condition follows the same failure pattern and is 48.4% at split 32. Because references are independently encoded with positions restarting at zero and selected all-memory payloads are not a causally encoded prefix, practical native-all is not equivalent to the exact historical-K/V reinsertion test. The experiment therefore validates sparse oracle transport under moderate fragmentation, not arbitrary partition invariance or learned routing.

13 Evidence-Anchored Natural-Text Native-KV Probes

The next gate retains the controlled target while replacing character-level distractors with natural documents from HotpotQA and QASPER [12, 3]. These are deliberately transport probes, not benchmark QA scores. For HotpotQA we use balanced yes/no questions, the supplied supporting-fact sentences, and distractor context. For QASPER we use balanced yes/no annotations and their human evidence spans from the official v0.3 corpus. Each example contains 63 fixed source units. The first two units are an explicit `AnswerCode yes|no` anchor; the remaining units contain evidence and sampled document words. The local tail asks only for that code. The original question, evidence, paper/document identity, and source metadata remain in the machine-readable record.

This construction separates two questions that a tiny model otherwise confounds. It does not ask whether a two-layer decoder trained from scratch can perform multi-hop or scientific-document QA. It asks whether a normally trained SelfAttention checkpoint can use a causally necessary token in natural-text context, whether sparse native K/V recovers that benefit after the source is displaced, and whether gist routing finds the known anchor. A direct HotpotQA pilot without the anchor failed this gate: held-out dense-context loss was worse than tail-only loss, making RCB uninterpretable. We therefore reject that pilot rather than attribute language-model failure to PRA.

Training and evaluation examples come from disjoint official splits. Across total split counts $\{2, 3, 5, 8, 16, 32, 64\}$, source text, local tail, answer token, tokenizer, checkpoint, and example order are identical; only reference boundaries vary. We report the same five paired seeds $\{1, 7, 21, 42, 87\}$ and the same native conditions as in the synthetic gate. No PRA transport or routing parameter is trained after the full-context SelfAttention checkpoint is frozen and converted.

13.1 HotpotQA-Derived Result

The HotpotQA-derived probe uses 4,000 balanced training examples and 64 examples from the disjoint validation split. A two-layer, width-128 decoder is trained for 150 optimizer steps with an 8,000-token BPE vocabulary. Table 4 reports five-seed means. Dense full context solves the target with loss 0.0027, while tail-only loss is 1.7038. The denominator required by RCB is therefore large and positive rather than an artifact of a low-dependency target.

Table 4: HotpotQA-derived answer-code transport probe, five-seed means. RCB is recovered context benefit; active is the oracle active token-K/V fraction.

Split	SA full	SA tail	Native all	Oracle	Routed	Shuffled	Oracle RCB	Routed RCB	Active
2	.0027	1.7038	.0029	.0029	.0029	4.0042	1.000	1.000	1.000
3	.0027	1.7038	.0249	.0035	.3835	2.5983	.999	.807	.524
5	.0027	1.7038	.0408	.0051	.5944	2.1552	.997	.653	.284
8	.0027	1.7038	.0432	.0056	.6168	2.0606	.997	.639	.202
16	.0027	1.7038	.0786	.0058	.6179	1.9974	.996	.641	.129
32	.0027	1.7038	.1213	.0060	.6812	2.2251	.996	.634	.092
64	.0027	1.7038	.9541	.8045	.9239	1.2470	.546	.405	.092

The decomposition localizes the result. At split 32 the mean transport gap is 0.119, the sparse gap is -0.115, and the routing gap is 0.675. Independent all-memory composition has accumulated partition cost, while selecting the two anchor units removes most of that cost; routing, not oracle transport, is the dominant remaining error. The oracle content-causality gap at split 32 is 2.219 loss units. Across all splits, the high-dependency tertile has mean oracle RCB 0.903, routed RCB 0.749, and shuffled RCB 0.309. These per-example ratios are more variable than the aggregate-loss ratios and are reported with medians in the machine-readable artifact.

Split 64 is a clear boundary. Oracle loss rises to 0.8045 and oracle RCB falls to 0.546; individual seeds range from near-dense recovery to losses above the tail baseline. This is not an active-budget increase—the mean active fraction remains 0.092—but a fine-fragment encoding/composition failure. The same condition also reduces the all-memory versus oracle contrast, so the 64-way point should not be read as evidence that routing has suddenly improved.

The timing fields are diagnostic rather than a hardware benchmark: these runs use a tiny CPU model and a Python cache builder. At split 32, oracle materializes about 10 active tokens and transfers 4,096 K/V bytes per example on average, versus about 110 active tokens and 207,872 bytes for all-memory. Mean measured example time is 10.7 ms for oracle and 853 ms for all-memory;

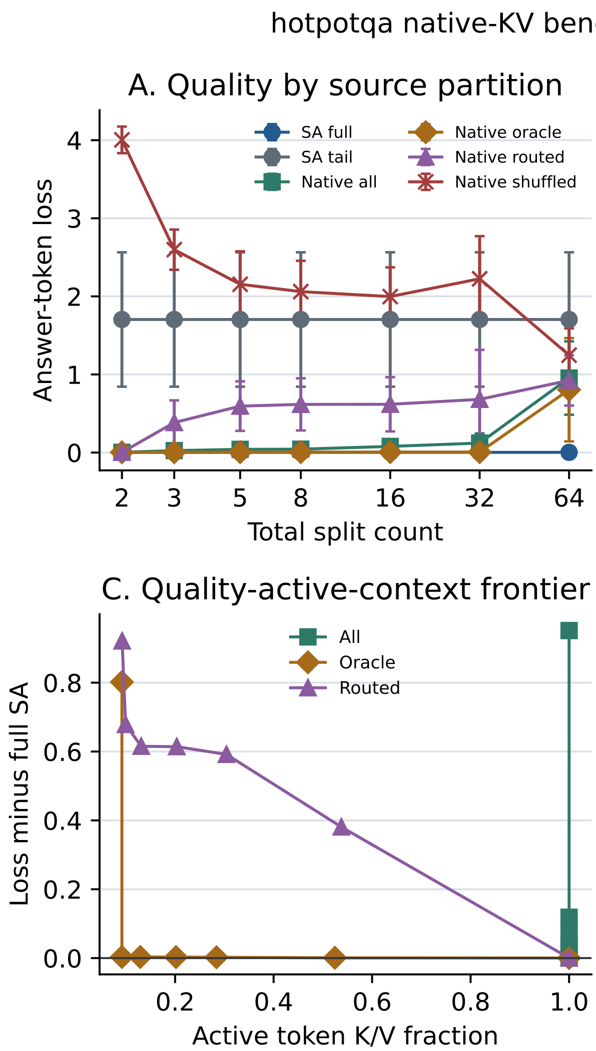

Figure 4: HotpotQA-derived quality, RCB, active-context frontier, and error decomposition. Oracle native K/V tracks dense context through split 32 while routed selection leaves a substantial oracle-to-routed gap. Error bars are population standard deviations over five paired seeds.

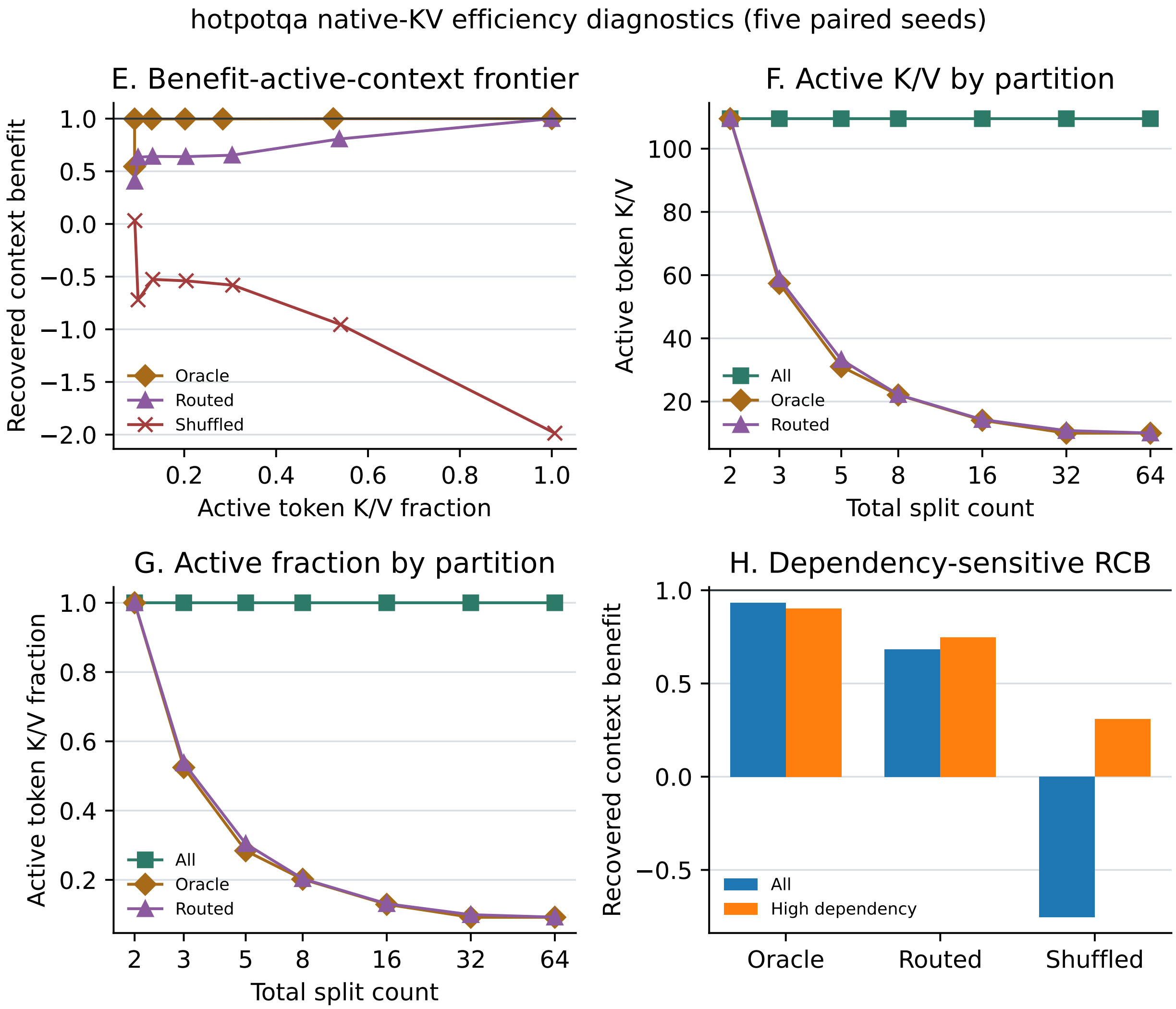

Figure 5: HotpotQA-derived active-K/V diagnostics. Through split 32, oracle RCB remains near one as active fraction falls to 0.092. The high-dependency stratum is formed from data-driven dependency-gain tertiles, not a hand-selected threshold.

at split 64 it is 12.5 ms versus 1.70 s. These numbers include prototype overhead and do not establish production speedup, but they verify that the quality-active-context frontier corresponds to measured materialization and transfer reductions rather than a label-only budget.

13.2 QASPER-Derived Result

The QASPER-derived probe repeats the protocol on scientific papers and human-annotated evidence spans. It uses 200 balanced training probes and 64 probes from the disjoint development split, the same two-layer width-128 architecture and 150-step schedule, and a BPE vocabulary learned only from the training corpus. Table 5 reports five-seed means.

Table 5: QASPER-derived answer-code transport probe, five-seed means. Active is the oracle active token-K/V fraction.

Split	SA full	SA tail	Native all	Oracle	Routed	Shuffled	Oracle RCB	Routed RCB	Active
2	.0026	1.8545	.0035	.0035	.0035	4.8820	.999	.999	1.000
3	.0026	1.8545	.0610	.0036	.2977	3.4029	.999	.790	.525
5	.0026	1.8545	.0925	.0037	.4924	2.6978	.999	.661	.281
8	.0026	1.8545	.1267	.0037	.6078	2.6359	.999	.572	.192
16	.0026	1.8545	.1769	.0037	.6640	2.2348	.999	.542	.117
32	.0026	1.8545	.1612	.0039	.5550	2.7351	.999	.602	.088
64	.0026	1.8545	.8835	.5047	1.0982	1.1819	.748	.338	.088

QASPER reproduces the HotpotQA-derived pattern with a different source distribution. At split 32, the transport gap is 0.159, sparse gap is -0.157, and routing gap is 0.551. The oracle content-causality gap is 2.731. High-dependency examples have mean oracle, routed, and shuffled RCB of 0.968, 0.823, and 0.391, respectively. At split 64, oracle loss rises to 0.5047 and oracle RCB falls to 0.748, while routed RCB falls to 0.338. The failure is milder than HotpotQA's mean oracle degradation but remains strongly seed-dependent.

At split 32, oracle activates about 10 tokens and transfers 4,096 K/V bytes per example, versus 114 tokens and 217,920 bytes for all-memory. Mean prototype times are 22.1 ms and 604 ms, respectively; at split 64 they are 25.2 ms and 1.26 s. Because all quality runs use CPU and a Python reference runtime, these values validate accounting and direction, not optimized systems performance. Together, the two natural-text probes support sparse native transport and expose a repeatable routing gap. They still do not establish unrestricted HotpotQA or QASPER question answering, because the answer-code anchor makes the causal dependency controlled and explicit.

14 Historical Version-1 Results

Every result from this section through the corrected five-seed reference adaptation study uses the optional cross-attention transport. The values remain informative about adaptation and preservation, but they are not evidence for native-KV equivalence, sparsity, or inference economics.

Tables 6 and 7 report population means and standard deviations over seeds 1 and 7. They are real report artifacts, but the reference runs predate the corrected hierarchy. They routed a single raw summary embedding per whole reference, reused a batch-collapsed query in an early path, and materialized whole-reference K/V. Accordingly, only the no-memory language-model results transfer directly to the current implementation; reference-conditioned values are engineering history, not evidence for corrected PRA.

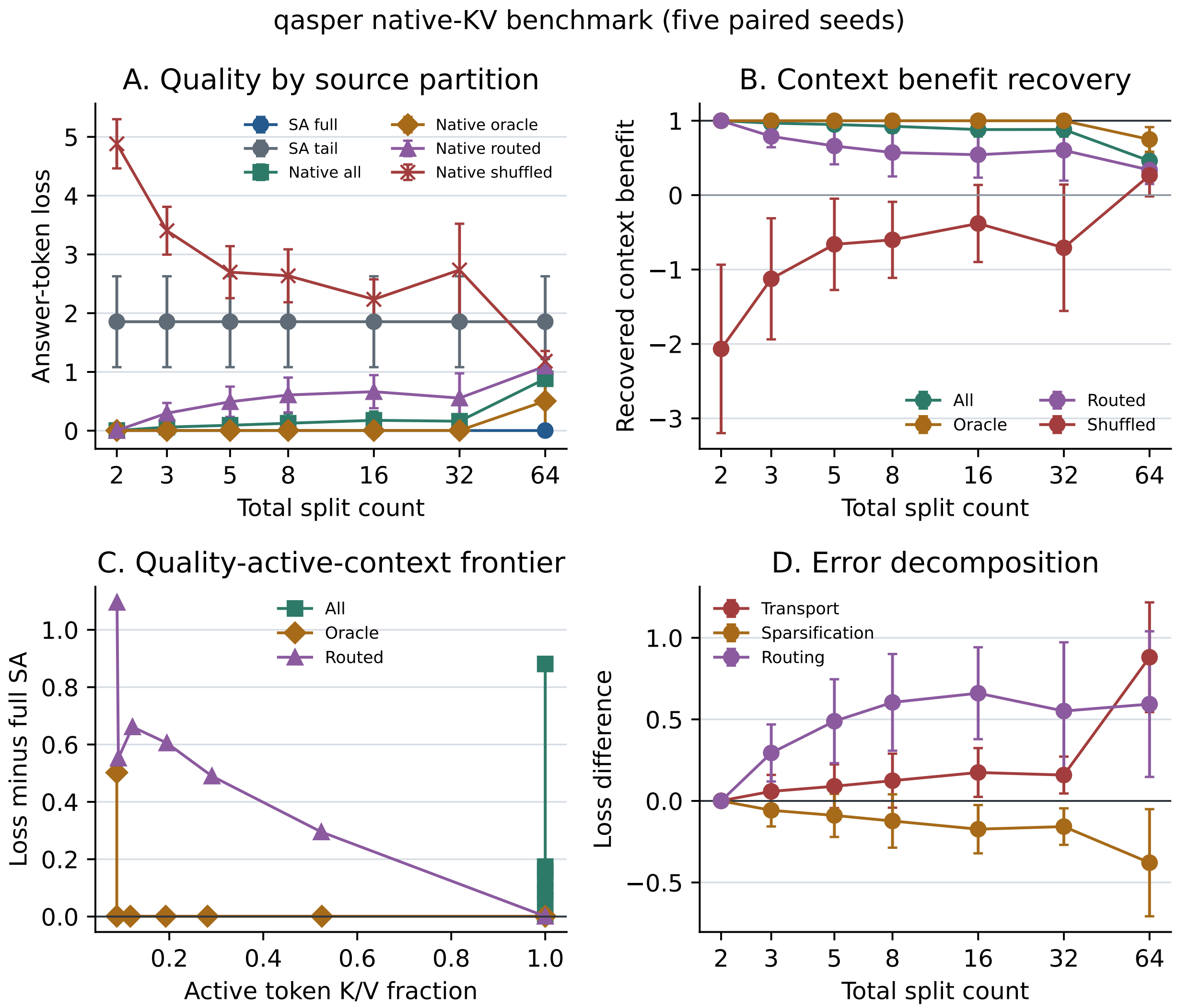

Figure 6: QASPER-derived quality and error decomposition over five paired seeds. Oracle native K/V preserves dense-context quality through split 32, while all-memory composition and routed selection accumulate distinct errors.

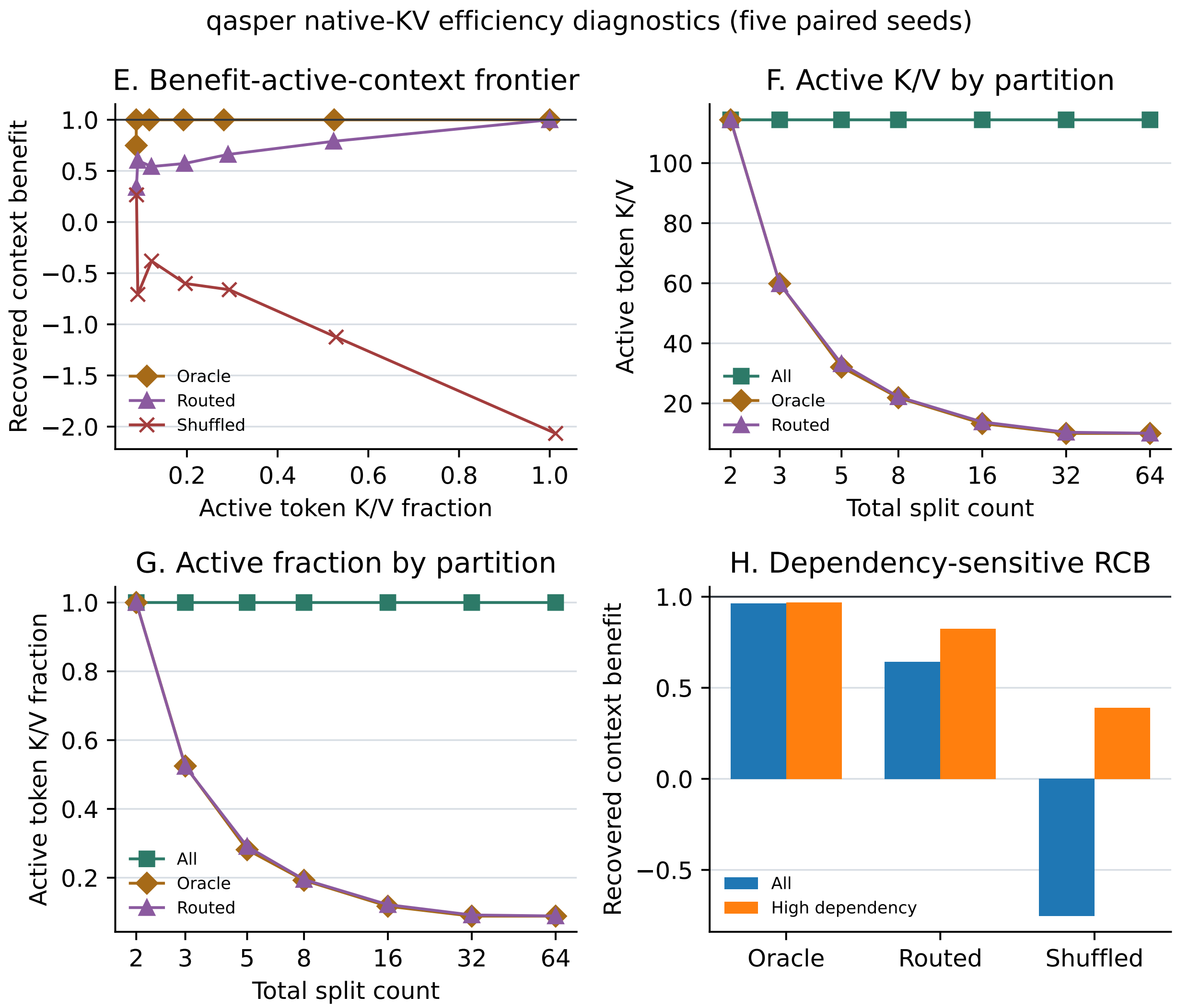

Figure 7: QASPER-derived active-K/V diagnostics. Oracle RCB remains 0.999 as active fraction falls to 0.088 through split 32; the 64-way point again marks a fragmentation limit.

Table 6: Phase-1 WikiText-2 language modeling at a matched 524,288-token budget.

Architecture	Parameters	Test loss	Perplexity	Token acc.	Train s
SelfAttention	4,216,320	$4.7691 \pm .0036$	$117.82 \pm .42$	$.1264 \pm .0006$	94.07 ± 1.43
Full PRA	4,479,488	$4.7457 \pm .0011$	$115.09 \pm .13$	$.1250 \pm .0012$	90.86 ± 1.13
Hybrid PRA	4,347,904	$4.7574 \pm .0017$	$116.44 \pm .19$	$.1271 \pm .0004$	$95.66 \pm .04$

All three architectures learn an ordinary token-prediction function and finish within 0.024 test-loss units of one another. This supports the narrow claim that a PRA block behaves comparably to the local baseline when no external memory is present at this scale and budget. It does not establish equivalence under scaling, and the models are not parameter matched.

Table 7: Phase-2 reference-conditioned answer-token loss and plain-text retention. Lower is better.

Architecture	Valid	Disabled	Shuffled	Plain Δ	Train s
SelfAttention	$4.7791 \pm .0734$	$4.7791 \pm .0734$	$4.7791 \pm .0734$	+0.1859	$44.82 \pm .45$
Full PRA	$4.6749 \pm .0607$	$4.7914 \pm .0683$	$4.6894 \pm .0563$	0.0000	45.13 ± 2.83
Hybrid PRA	$4.7111 \pm .0356$	$4.7953 \pm .0284$	$4.7233 \pm .0351$	0.0000	$31.14 \pm .02$

In that implementation, disabling reference memory increased mean loss by 0.1165 for full PRA and 0.0841 for hybrid PRA, while shuffling increased loss by 0.0145 and 0.0122. These deltas show that the old residual branch changed predictions, but cannot establish correct per-example chunk selection under the corrected semantics. They motivate, but do not substitute for, rerunning counterfactuals with the new reference/chunk traces.

Plain-text reevaluation after phase 2 exactly reproduces the phase-1 losses for full and hybrid PRA because frozen base parameters and zero-initialized isolated output projections protect the local path. The SelfAttention control's mean plain loss rises from 4.7691 to 4.9550, a 0.1859 increase corresponding to about a 20% perplexity increase. This demonstrates preservation by the chosen training strategy, not an inherent guarantee of PRA: the control updates all parameters while PRA updates only reference-specific projections.

Phase-1 mean synchronized validation times are 1.07–1.14 seconds and mean wall-clock times are 97.0–102.0 seconds. Phase-2 mean synchronized training times are 31.1–45.1 seconds, with 46,301.5 mean processed prompt/answer tokens and 512 optimizer steps. These historical timings predate the batch-aware prompt-forward path and must not be used as measurements of its speedup. New reports should separate reference-cache construction, row-local routing, the single prompt forward, and bucketed memory attention.

15 Corrected Five-Seed Controlled Follow-Up

The corrected study executes 65 CUDA runs: 25 ordinary-language runs (five architecture/capacity groups times five seeds) and 40 fixed-reference runs. Every condition uses paired model seeds $\{1, 7, 21, 42, 87\}$, the same fixed tokenizer and data split, and the corrected per-row, per-layer chunk router. Tables report means and population standard deviations over seeds. With five pairs, a two-sided exact sign-flip test cannot attain $p < 0.05$ when every paired difference has the same sign; its minimum is $p = 0.0625$. We therefore emphasize paired effect direction and magnitude rather than binary significance.

Table 8: Five-seed plain WikiText-2 follow-up at a 1,048,576-token budget. Active time is synchronized GPU training time.

Architecture	Parameters	Test loss	Train tok/s	Active s
SelfAttention, same width	4,216,320	$4.5704 \pm .0076$	5411 ± 125	193.9 ± 4.5
Full PRA	4,479,488	$4.5034 \pm .0163$	5623 ± 110	186.4 ± 3.4
Hybrid PRA	4,347,904	$4.5500 \pm .0112$	5438 ± 123	192.9 ± 4.4
SelfAttention, matched to full	4,478,976	$4.5855 \pm .0347$	–	–
SelfAttention, matched to hybrid	4,347,648	$4.5715 \pm .0082$	5657 ± 38	185.4 ± 1.2

Full PRA is 0.0820 loss units better than its nearly parameter-matched SelfAttention control, and hybrid PRA is 0.0215 better than its matched control; every paired seed has the same sign. The exact paired p-value is nevertheless 0.0625 in both comparisons. These results support ordinary-language compatibility and motivate replication; they do not establish architectural superiority or statistical equivalence. Relative to the historical 524,288-token pilot, full-PRA mean test loss falls from 4.7457 to 4.5034, a 0.2423 reduction. The longer tested pretraining schedule helps, but it does not identify an optimum. One matched-full run resumed after a host-side wrapper timeout, so its quality result is retained while its aggregate timing is omitted from the table.

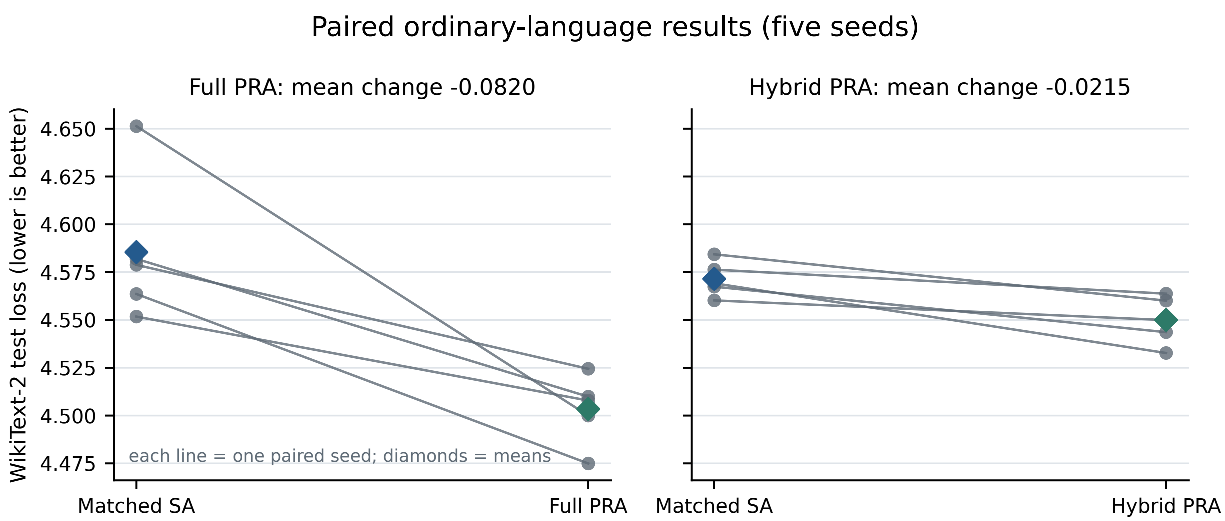

Figure 8: Paired ordinary-language losses for the historical cross-attention-capable backbones. Each line connects one seed to its nearly parameter-matched SelfAttention control; diamonds mark five-seed means. External memory is absent in these runs. The directional result is encouraging but underpowered ($p = 0.0625$), and it is not a native-KV quality measurement.

Table 9: Corrected-router reference-task training order. Values are five-seed means; lower loss is better.

Architecture	Order	Valid	Disabled	Plain after	Top-1	Train tok/s
SelfAttention	scratch	6.3459	6.3459	6.4787	–	1615
SelfAttention	joint	4.9230	4.9230	4.7125	–	–
Full PRA	scratch	6.3557	6.3754	6.4948	.4212	983
Full PRA	frozen path	4.6720	4.6895	4.5034	.4221	1244
Full PRA	joint	4.9129	4.8938	4.6809	.4269	1052
Hybrid PRA	scratch	6.3642	6.3846	6.5140	.4231	1157
Hybrid PRA	frozen path	4.7296	4.7413	4.5500	.3981	1622
Hybrid PRA	joint	4.9196	4.9125	4.7062	.4077	1141

Pretraining strongly improves final reference-task loss. Frozen-path training improves over scratch by 1.6837 for full PRA and 1.6346 for hybrid PRA; direct joint training improves by 1.4429 and 1.4447. Each paired effect has the same sign in all five seeds. Frozen training exactly reproduces the corresponding 4.5034 and 4.5500 plain losses because the base is frozen. Direct joint tuning instead increases plain loss by 0.1775 for full PRA and 0.1562 for hybrid PRA. It also finishes 0.2408 and 0.1899 loss units worse than frozen-path adaptation. Direct joint fine-tuning is therefore not sufficient under this budget. The hybrid does not benefit more from staging than full PRA: its scratch-to-frozen improvement is smaller.

Table 10: Counterfactual losses after corrected frozen-path training. Five-seed means.

Architecture	Valid	Disabled	Shuffled	Irrelevant	Oracle
Full PRA	4.6720	4.6895	4.6769	4.6723	4.6702
Hybrid PRA	4.7296	4.7413	4.7304	4.7304	4.7279

The frozen memory branch contributes relative to disabling it, by 0.0175 loss for full PRA and 0.0117 for hybrid PRA. Correct content is not yet the source of that advantage: shuffled-reference penalties are only 0.0048 and 0.0008, irrelevant penalties are below 0.001, and oracle references improve loss by only 0.0018 and 0.0017. Full-PRA test top-1 is 0.4221 ± 0.0156 and hybrid top-1 is 0.3981 ± 0.0397, against a candidate-count-weighted chance rate of 0.4202. Periodic curves remain flat rather than showing faster selection after pretraining. Thus pretraining improves language competence and adaptation loss, while the current unsupervised discrete router does not learn generalized reference selection.

The timing counters also separate optimization conclusions from systems claims. Across uninterrupted runs, full-PRA scratch, frozen, and joint training process approximately 983, 1,244, and 1,052 tokens/s; hybrid variants process 1,157, 1,622, and 1,141 tokens/s. Frozen training is faster because only the memory output path receives gradients. These are single-device prototype measurements with per-example reference encoding and no persistent cache reuse. They characterize this implementation, not an efficiency advantage over long-context baselines.

16 Post-Refactor Fixed-Split Smoke Study

After separating the generic training engine into `src/common`, we reran the standalone synthetic workflows, plain WikiText-2 smoke notebooks, and a corrected-router fixed-split matrix. The

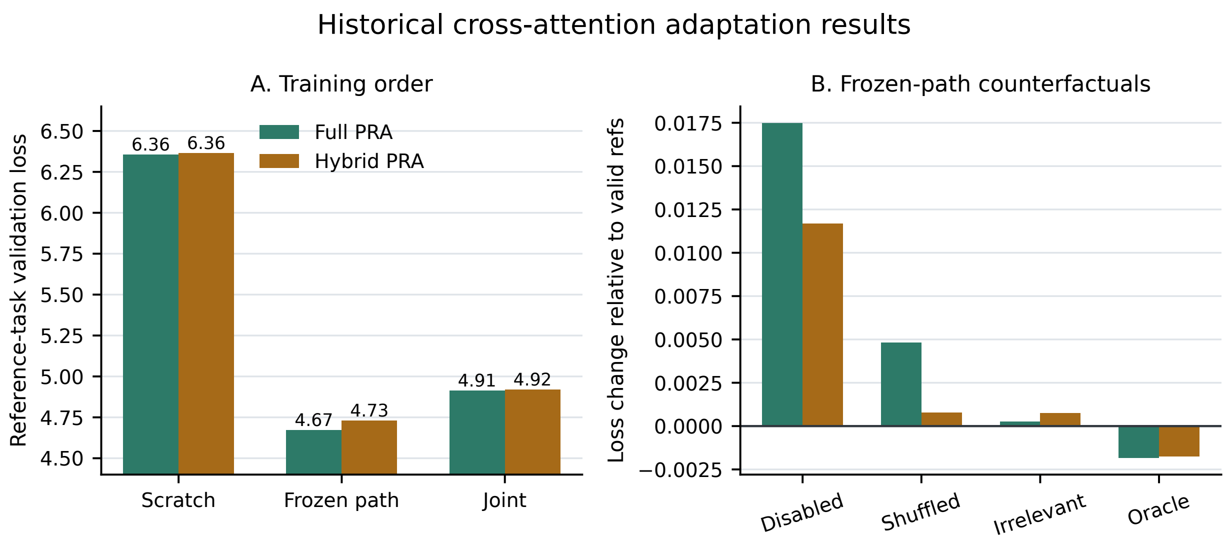

Figure 9: Historical cross-attention adaptation results. Left: frozen-path adaptation finishes substantially below scratch and direct joint tuning for both PRA placements. Right: frozen-path loss changes relative to valid references; positive values are worse. Disabling memory has a visible cost, but shuffled and irrelevant changes and oracle gains are small, so the branch effect is not evidence of content-causal retrieval.

matrix uses 32 WikiText-2 source entries, paired model seeds {1, 7, 21, 42, 87}, a fixed data-generation and split seed of 1729, one shared 1,331-token BPE vocabulary, four layers, width 256, context length 128, batch size 2, and eight optimizer steps. Split count means total text parts including the prediction tail: split 2 exposes one earlier part, while split 5 exposes four. Source entries, tokenizer, optimizer-step budget, and sequence length are shared across the 20 runs. The target span and processed-token count nevertheless change with the partition, so the comparison is a pipeline and scaling diagnostic rather than an isolated estimate of split-count causality.

Table 11: Post-refactor CUDA smoke matrix. Values are population mean ± standard deviation over five paired model seeds. $\Delta_{\text{use}} = L_{\text{disabled}} - L_{\text{valid}}$ and $\Delta_{\text{content}} = L_{\text{shuffled}} - L_{\text{valid}}$.

Architecture	Splits	Tokens	Test loss	Δ_{use}	Δ_{content}	Train s
SelfAttention	2	903.0	7.1844 ± 0.0350	0	0	0.581 ± 0.068
SelfAttention	5	1,168.4	7.0881 ± 0.0646	0	0	0.884 ± 0.085
Full PRA	2	903.0	7.2207 ± 0.0310	0.0189 ± 0.0030	0.0002 ± 0.0006	0.959 ± 0.218
Full PRA	5	1,168.4	7.0357 ± 0.0542	0.0312 ± 0.0048	-0.0006 ± 0.0009	1.443 ± 0.224

The SelfAttention invariance across valid, disabled, and shuffled conditions is a useful control. Every PRA seed has a positive disabled-path gap in both partition regimes, so memory-branch activation is reproducible rather than a single initialization artifact. Correct content is not. Mean shuffled-reference changes are effectively zero, and split-5 reference top-1 is 0.150 ± 0.102, below the 0.25 uniform-candidate rate. Split-2 top-1 is trivially 1.0 because only one candidate exists. Split 5 raises mean synchronized training time by about 52% for both architectures while increasing processed target-bearing tokens by about 29%. PRA is about 63–65% slower than SelfAttention in these runs. Because each run lasts less than two seconds, startup, synchronization, unequal parameter counts, and unequal token totals preclude quality or systems superiority claims. The defensible contribution is narrower: the corrected batched path, counterfactual evaluator, paired-seed aggregation, split metadata, timing counters, notebook outputs, and HTML/JSON reports execute together without regression.

16.1 Reproducibility and Artifact Scope

Resolved per-seed HTML and JSON reports, plots, timing counters, checkpoints, and aggregate reports are generated under `out/reports`; the consolidated study is in `out/reports/phase1_followup_v2_5seed_findings`. The aggregate computes population mean and standard deviation without discarding individual seed values and records the fixed data-generation and split seed. Active GPU training time is reconstructed from synchronized batch durations. Content hashes, peak allocated memory, matched token and trainable-capacity budgets, longer training, and independent replication remain necessary before treating the result as publication-grade.

17 Relationship to Prior Work

The prototype builds on the transformer attention architecture [10]. It is related to sparse and long-context attention models such as Longformer, BigBird, Reformer, and Routing Transformer [1, 13, 5, 9], but the central question differs: PRA studies whether external context can remain addressable until needed rather than being placed in a single long sequence.

It also relates to memory-augmented and retrieval-conditioned models. Compressive Transformers maintain compressed memories over earlier activations [8]. Memorizing Transformers retrieve nearest-neighbor memories [11]. RETRO conditions generation on retrieved chunks from a large database [2]. RAG and REALM connect generation to non-parametric retrieval [7, 4]. PRA differs in its emphasis on explicit handles, URI-addressed references, recursive anchors, and per-layer cache construction.

Finally, the implementation anticipates runtime concerns raised by serving systems such as vLLM's PagedAttention [6]. A mature PRA system would require cache admission, sharing, eviction, batching, and memory safety policies.

18 Safety and Systems Correctness

Reference memory expands the model's attack and failure surface. A malicious or compromised reference can poison summaries, embed adversarial instructions, or exploit trusted URI namespaces. URI resolution must therefore preserve source, content digest, version, authorization decision, and resolver identity. Entries from different trust domains should not be merged into an anonymous K/V pool, and outputs should retain enough provenance to reconstruct which objects were resident.

Freshness is part of correctness. A cache key based only on URI can return stale memory after content or model parameters change. Persistent implementations should bind entries to content version, model/adapter identity, layer, representation format, and policy domain. Revocation must invalidate both object lookup and already encoded resident representations.

Batching creates a confidentiality boundary. The corrected prototype constructs a cache per example after a batch-union design exposed cross-example references, then places those caches behind a row-indexed wrapper for one prompt forward. Optimized kernels must preserve equivalent masks and should include adversarial tests proving that logits for one sequence are invariant to another sequence's private cache. Cache reuse across users or tenants requires authorization-aware deduplication.

Recursive expansion can amplify work. The standalone builder now bounds depth, child count, total references, and encoded tokens; detects cycles; exposes missing-reference and overflow policies; and prevents partial entries from becoming searchable. It also fingerprints content, model, tokenizer, and routing configuration. Authorization, latency/byte budgets, revocation, cryptographic provenance, and durable cache invalidation remain deployment requirements beyond the in-memory resolver.

19 Limitations

The current prototype and follow-up have important limitations. Native-KV transport now has controlled quality, active-K/V, transfer-byte, and prototype-timing sweeps, but it does not yet have optimized-kernel or production-hardware evaluation; historical K/V reuse has not yet been compared exhaustively with independent chunk re-encoding across position policies. The HotpotQA- and QASPER-derived tasks inject a fixed answer code and therefore test transport over natural text rather than unrestricted multi-hop or scientific-document QA. Their fixed anchor favors an oracle and does not establish semantic routing. The corrected-router cross-attention smoke matrix reaches the planned minimum of five paired model seeds, but eight optimizer steps and 32 source entries make it an engineering pilot rather than a benchmark. Parameter matching controls ordinary-language capacity in the historical study, while the smoke matrix does not match trainable parameters across architectures. The fixed version-2 reference dataset marks an adjacent source part as relevant; it does not provide human-verified causal evidence spans, and the model can reduce continuation loss without learning candidate selection. The near-chance selector and weak counterfactual deltas leave open how much of the historical adapted-memory gain is content independent.

The corrected selector is still heuristic and receives no auxiliary relevance loss. It uses a projected last-valid-token query rather than explicit reference-token positions, and hard top-k selection remains nondifferentiable. Most gist modes and full token K/V are detached by default; only the optional GRU gist mode supplies a trainable cache-summary path. Recursive construction is deterministic and child-first, not a learned iterative policy. The training adapter now batches the prompt forward, but reference encoding and routing still iterate over row-private caches. Multi-gist modes and the implicit prompt-head path have focused tests but no controlled quality, routing, or timing study. In particular, prompt-head encoding can relocate rather than remove computation, and generated-token rollover is absent. The 20-run fixed-split matrix varies model initialization but keeps one generated dataset and split; summary modes, oracle chunks, full-reference transport, recursion, and dynamic bucketing remain unit- and CUDA-smoke-tested rather than evaluated in a controlled training study. Finally, one tiny model scale cannot establish scaling or efficiency behavior. No broad experimental superiority is claimed.

20 Conclusion

The standalone PyTorch PRA prototype now makes native K/V the default transport, converts trained SelfAttention weights without adding transport parameters, and preserves the former cross-attention path for reproducibility. Layer-level historical-K/V reinsertion and whole-model disabled-memory conversion pass numerical equivalence tests. The fixed-source synthetic gate adds content-causal quality evidence. The HotpotQA- and QASPER-derived probes show the desired scaling relation through 32 partitions: oracle RCB remains 0.996 and 0.999 while active token K/V falls to 0.092 and 0.088. Routed RCB of 0.634 and 0.602 leaves a measurable selector gap in both domains. Their 64-way degradation and native-all fragmentation curves rule out a broad partition-invariance claim. The earlier cross-attention studies establish useful adaptation controls but fail their content-causality gate. The immediate priorities are to replace answer-code anchors with genuine QA targets using a capable pretrained backbone, train routing against the demonstrated oracle ceiling, and compare native cache semantics across positional schemes. End-to-end production efficiency remains unclaimed. Pretrained Hugging Face integration will additionally need to preserve rotary positions, grouped-query caches, and layer-specific cache layouts.

A Reference-Memory Forward Pass Pseudocode

```
row_caches = []
for sample in minibatch:
    cache = empty_row_cache()

    def ensure_cached(uri, depth, shared_budget, path):
        if uri in path: apply_cycle_policy(); return
        if cache.state(uri) == READY: return
        resolved = resolver.resolve(uri)
        cache.mark_building(uri)
        shared_budget.consume_reference(uri)

        # Explicit local table only; children become searchable first.
        for child_uri in resolved.reference_table.values():
            ensure_cached(child_uri, depth + 1,
                          shared_budget, path + [uri])

        chunks = partition(resolved.token_ids, routing_config)
        hidden = embed(resolved.token_ids)
        entry = CacheEntry(uri=uri, chunks=chunks)
        for layer in model.layers:
            if layer.has_pra:
                for chunk in chunks:
                    K, V = layer.project_reference_kv(hidden[chunk.span])
                    gists = compute_gists(K, V, routing_config)
                    entry.add(layer.id, chunk.id, K, V, gists)
            hidden = layer(hidden, use_ready_child_memory=True)

        cache.put_ready(detach_by_policy(entry))

    split = split_exact_ids(sample.input_ids, direct_limit)
    if split.implicit_ids:
        register_request_local_head(cache, split.implicit_ids,
                                    uri="pra://implicit/prompt/head")

    for handle in sample.references:
```

```
        ensure_cached(handle.uri, 0, shared_budget(), [])

    row_caches.append(cache)
    direct_rows.append(split.direct_ids)

batch_cache = BatchCache(row_caches)  # query row i sees row_caches[i]
model.set_pra_cache(batch_cache)
input_ids, valid_mask = pad(direct_rows)
logits = model(input_ids, attention_mask=valid_mask)
loss = cross_entropy(logits, labels, ignore_index=PAD)
```

The preparation loop is intentional: cache identity is part of the example. It no longer implies one Transformer prompt forward per sample. The batch wrapper preserves row ownership during routing, and the attention implementation executes unequal selected memory lengths in masked buckets. A future vectorized cache builder must preserve the same example-to-cache mapping.

B Simplified PyTorch PRAttention

The following excerpt omits thresholds, score aggregation, summary modes, recursion, dropout, and shape utilities while retaining the corrected separation between routing gists and materialized token memory.

```
class PRAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.o_proj = nn.Linear(d_model, d_model)

    def forward(self, x, cache, layer_id):
        q = split_heads(self.q_proj(x), self.n_heads)
        k = split_heads(self.k_proj(x), self.n_heads)
        v = split_heads(self.v_proj(x), self.n_heads)

        # Each row routes independently using the projected last query.
        routing_q = merge_heads(q[:, :, -1:, :]).squeeze(1)
        selected_by_row = cache.hierarchical_search(
            routing_q, layer_id,
            top_k_references=kr,
            top_k_chunks_per_reference=kc,
        )
        memory_k, memory_v = materialize_selected_chunks(
            selected_by_row, layer_id
        )
        rows = []
```

```
for i, (mem_k, mem_v) in enumerate(zip(memory_k, memory_v)):
    # Memory is historical, so every local query can see it.
    all_k = cat([mem_k, k[i:i+1]], dim=2)
    all_v = cat([mem_v, v[i:i+1]], dim=2)
    scores = q[i:i+1] @ all_k.transpose(-2, -1)
    scores = scores / sqrt(self.head_dim)
    mask = native_causal_mask(x, mem_k.shape[2])
    scores = scores.masked_fill(mask, -inf)
    rows.append(softmax(scores, dim=-1) @ all_v)
return self.o_proj(merge_heads(cat(rows, dim=0)))
```

The actual implementation takes the original local-attention fast path when every selected memory is empty, preserving exact disabled-memory behavior. The optional cross-attention class path retains `memory_out` and `memory_alpha` only for legacy checkpoints and adapted-transport experiments.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[3] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021.

[4] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

[5] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[8] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[9] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, 2021.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[11] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[12] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

[13] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.